

Schiffe versenken / Battleships

Ungefähre Aufteilung:

- Frontend: Dennis
 - Win State definieren
 - Spielvorbereitung: P1 Schiffe platzieren
- Backend: Nuri
 - COM: Smartes Schießen + Platzieren

Allgemeines Konzept:

- Gespielt wird in einem beliebigen Terminal.
- Die beiden Spieler, ein menschlicher User (P1) und der Computer (COM), spielen abwechselnd.
- 10x10 Brett, (0-9, A-J, Koordinaten z.B. A0, F7, J9)
- Vor Spielbeginn legt jeder Spieler seine Flotte auf dem eigenen Gitter fest, wobei die Schiffe nicht horizontal, vertikal oder diagonal berühren dürfen – es muss mindestens ein Kästchen Wasser zwischen ihnen liegen.
 - COM: ...
- Platzieren von Schiffen erfolgt, indem der Spieler vorher auswählt, ob er Schiff X (evtl. groß -> klein) Horizontal oder Vertikal platzieren möchte. Anschließend gibt dieser Koordinaten ein. Schiff wird dann vom Kopf nach Rechts/Unten angezeigt und der Spieler muss bestätigen. Sobald alle Schiffe platziert wurden, beginnt das Spiel.
- P1 kann mit dem Eingeben von Koordinaten einen Schuss auf das Feld des COM verrichten. Das Terminal gibt das aktualisierte Spielbrett wieder und eine kurze Nachricht wie z.B. "Hit!", "Miss...", "Sunk!!" wird zurückgegeben.
- Bei einem Treffer eines Spielers (P1 oder COM) darf derjenige erneut schießen.
- COM wird zufällig auf ein Feld von P1 schießen
- Folgende Schiffe werden verwendet:
 - ein Schlachtschiff / Carrier (5 Felder),
 - ein Kreuzer(4 Felder),
 - zwei Fregatten (je 3 Felder)
 - ein Minensucher (2 Felder)

Arbeitsschritte:

- Spielbrett rendern
- Eingabe von Koordinaten handeln
- Schiffe platzieren
- Game State kontrollieren/aktualisieren (verschossen, getroffen, versunken, gewonnen)

Spielvorbereitung

- Schiffe platzieren (vorerst default Spielfelder in nem Array hard-codieren)

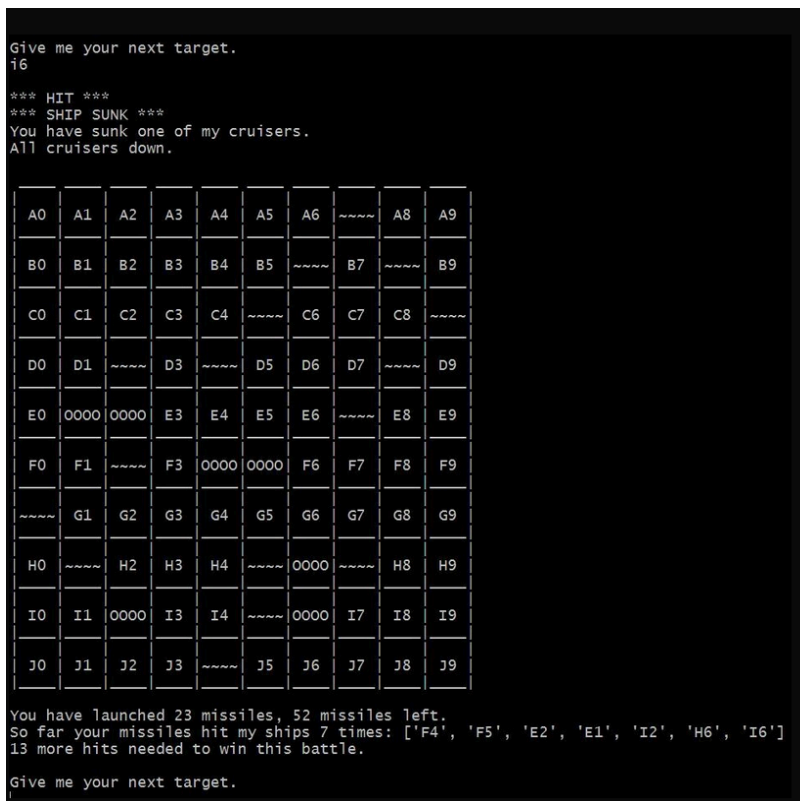
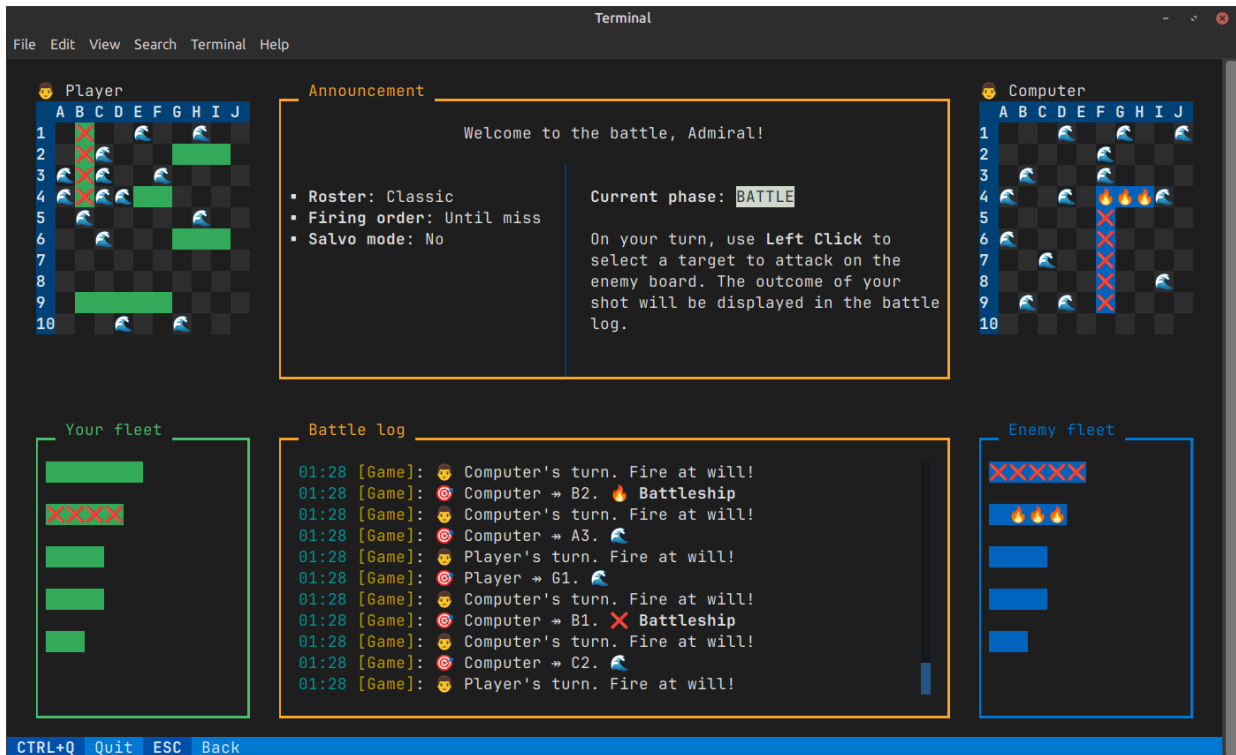
Spiel:

- Tracken, wer dran ist (P1 beginnt)
- Koordinate eingeben
- Gültige Eingabe?, wenn nicht, neu prompten
- Cell State an der Koordinate checken ("ist da ein Schiff oder nicht?")
- Spielbrett aktualisieren + rendern
- Nachricht im Terminal ausgeben (Hit, Miss, Sunk)
- Tracken, wer dran ist (Jetzt COM)

...

- COM: Selbes Prozedere, nur dass es vorerst ein zufälliges Feld wählt

Rendering



Erstes Mock-up:

```

  A B C D E F G H I J |  A B C D E F G H I J
0 # # # # # # # # # # | 0 ? ? ? ? ? ? ? ? ? ?
1 # # # # # # # # # # | 1 ? ? ? ? ? ? ? ? ? ?
2 # # # # # # # # # # | 2 ? ? ? ? ? ? ? ? ? ?
3 # # # # # # # # # # | 3 ? ? ? ? ? ? ? ? ? ?
4 # # # # # # # # # # | 4 ? ? ? ? ? ? ? ? ? ?
5 # # # # # # # # # # | 5 ? ? ? ? ? ? ? ? ? ?
6 # # # # # # # # # # | 6 ? ? ? ? ? ? ? ? ? ?
7 # # # # # # # # # # | 7 ? ? ? ? ? ? ? ? ? ?
8 # # # # # # # # # # | 8 ? ? ? ? ? ? ? ? ? ?
9 # # # # # # # # # # | 9 ? ? ? ? ? ? ? ? ? ?

Choose a target: B3
P1 -> B3: Miss!
depix@LOBOTOMIZER3000:~/uni/ppi/gruppe-c25/project$
```

Game Logic

- Spielbretter mithilfe von 2D Arrays speichern

Input/Output

- P1 gibt Koordinaten an, wobei A-J für die Spalten und 0-9 für die Zeilen angegeben werden
- Das Programm sollte Eingaben wie "A9", "a9", "9A", "9a" als gleich ansehen. Die Standard Terminal Ausgabe sollte aber immer im folgenden Format stattfinden:
 - "P1 -> A9: Hit!"
 - "COM -> B2: Miss!"

Platzieren der Schiffe

- Vorerst Schiffspositionen hardcodieren, dann als Spielvorbereitung in einem "Platzier-Modus", mit Eingabe der Koordinaten und Ausrichtung (vertikal, horizontal) der Schiffe implementieren.